

Dynamic Batch Formation Simulator

Experimental evaluation of fixed, dynamic, and continuous batching with real Qwen inference

Measured on an NVIDIA GeForce RTX 5050 Laptop GPU

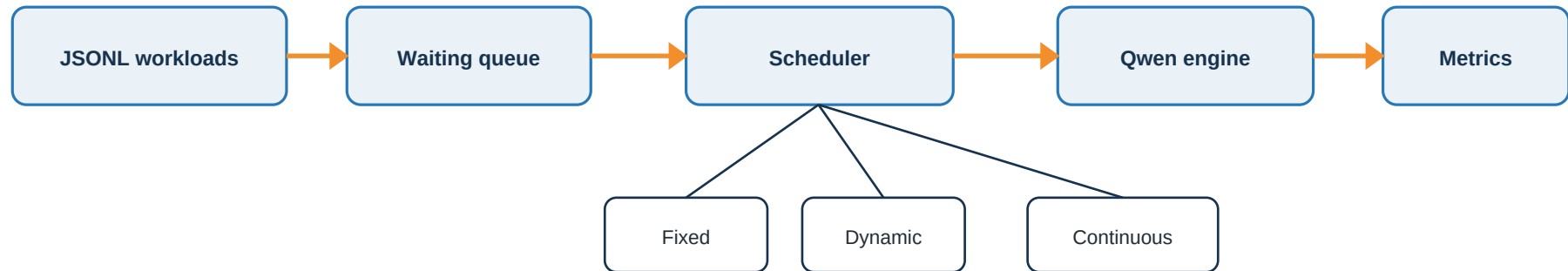
Dockerized benchmark | Two workloads | Three repetitions | Median results

Generated from the verified benchmark outputs in this repository.

1. Problem statement

This project implements fixed, dynamic, and continuous batching for real LLM inference inside Docker. It compares output-token throughput, request latency, time to first token (TTFT), and active batch size under dense and sparse request arrivals.

2. Architecture



A clean process runs one strategy at a time. Runs remain sequential. The strategy order rotates between repetitions to reduce order bias.

3. Batching strategies

Fixed. Waits for four requests, then runs that closed batch until every request finishes. The final batch may be smaller.

Dynamic. Starts a closed batch when four requests are ready or the oldest request has waited 25 ms.

Continuous. Checks for free slots after every token step and admits waiting requests as active requests finish.

4. Technology stack

Component	Version	Role
Python	3.11	Benchmark runner
PyTorch	2.8.0	CUDA 12.8 inference
Transformers	4.46.3	Qwen model and tokenizer
Matplotlib	3.9.2	Graphs

5. Experimental setup

Hardware: NVIDIA GeForce RTX 5050 Laptop GPU, 8151 MiB, driver 592.82. Model: Qwen/Qwen2.5-0.5B-Instruct in FP16. Batch size: four. Dynamic wait: 25 ms. Each strategy ran three times per workload.

Every strategy received the same prompts and max_new_tokens values. Each run completed 30 unique request IDs and generated 960 output tokens. Reported bars are medians, and error bars span the minimum and maximum run.

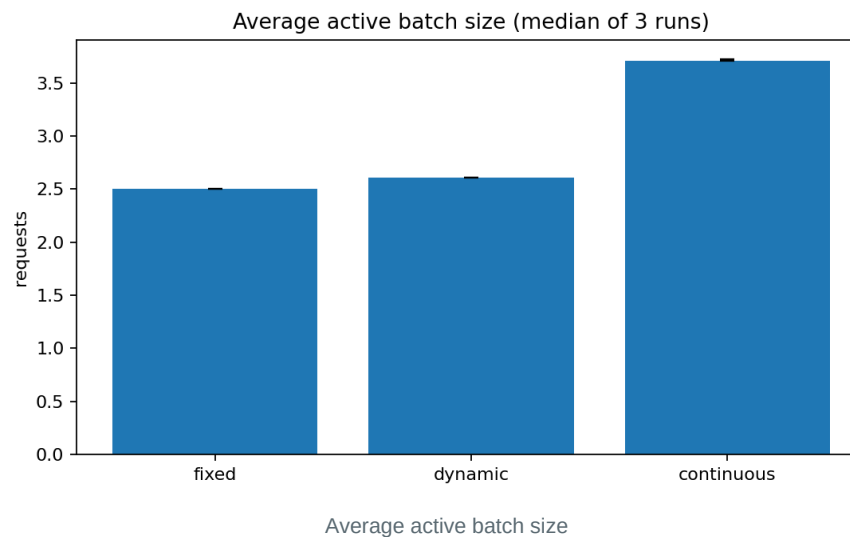
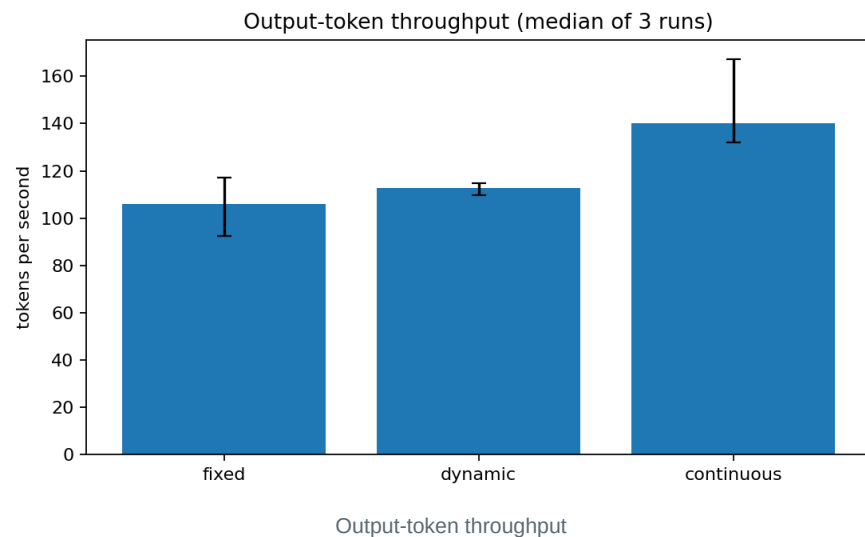
6. Workloads

Workload	Requests	Arrival interval	Output limits	Purpose
Dense	30	20 ms	16, 32, 48	Keep requests available
Sparse	30	400 ms	16, 32, 48	Expose queue waiting

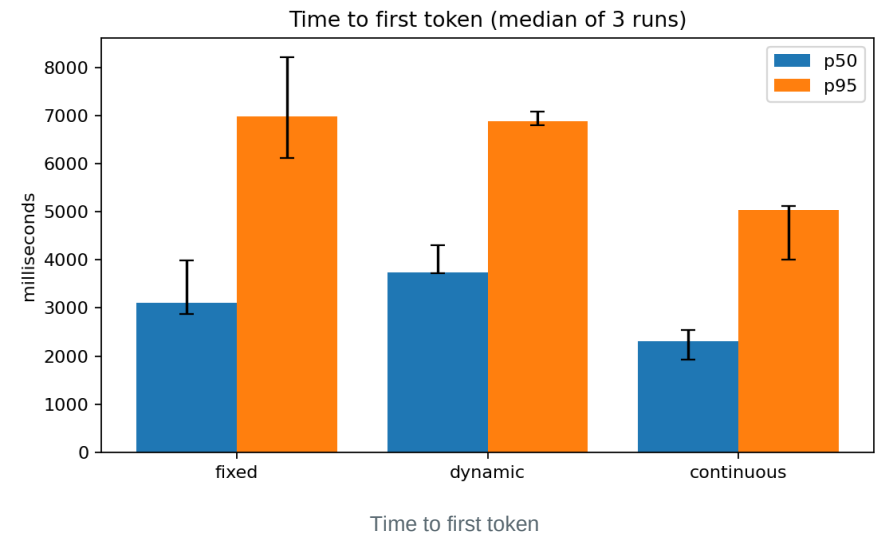
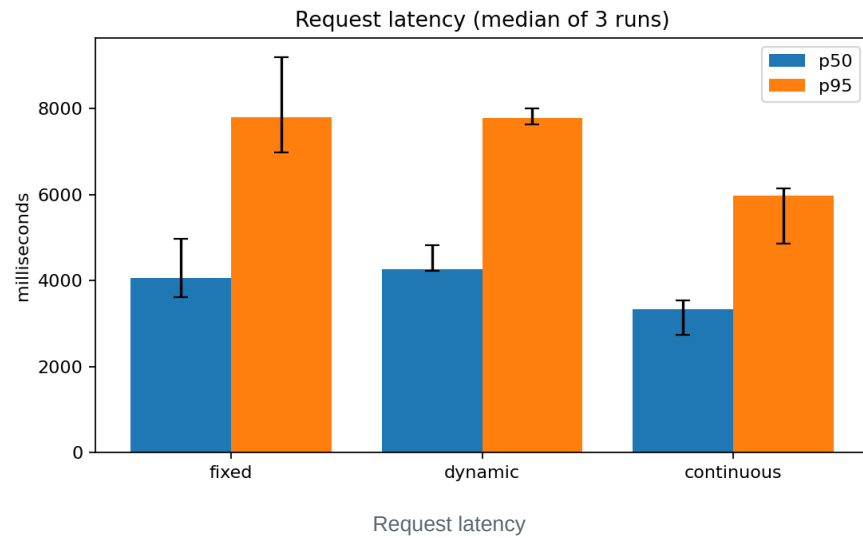
7. Dense workload results

Strategy	Tokens/s	Latency p50	Latency p95	TTFT p50	TTFT p95	Avg. batch
Fixed	106.04	4062.55	7806.35	3113.94	6973.02	2.50
Dynamic	112.58	4274.68	7780.91	3734.40	6879.27	2.61
Continuous	140.03	3329.73	5979.25	2316.10	5030.15	3.71

Continuous batching reached the highest throughput and lowest p50 and p95 latency. Its average active batch size of 3.71 shows that slot refilling kept the GPU busier. Dynamic throughput was slightly higher than fixed, while their latency ranges overlapped.



Dense latency detail

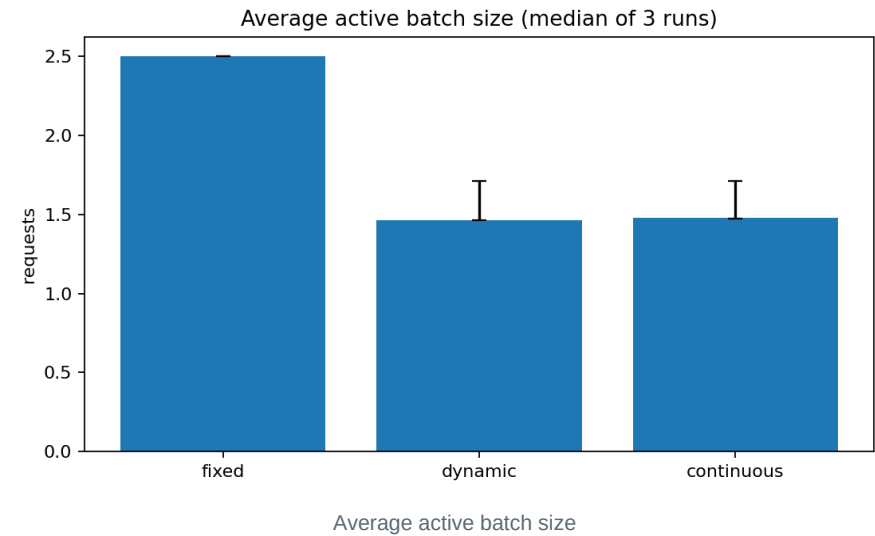
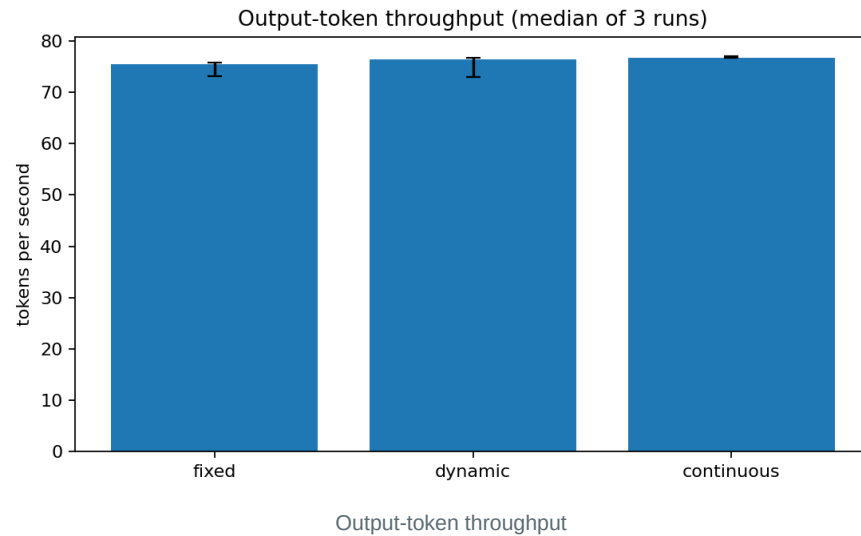


The dense workload builds a queue quickly. Closed batches keep later requests waiting until earlier batches finish. Continuous admission shortens both request latency and TTFT.

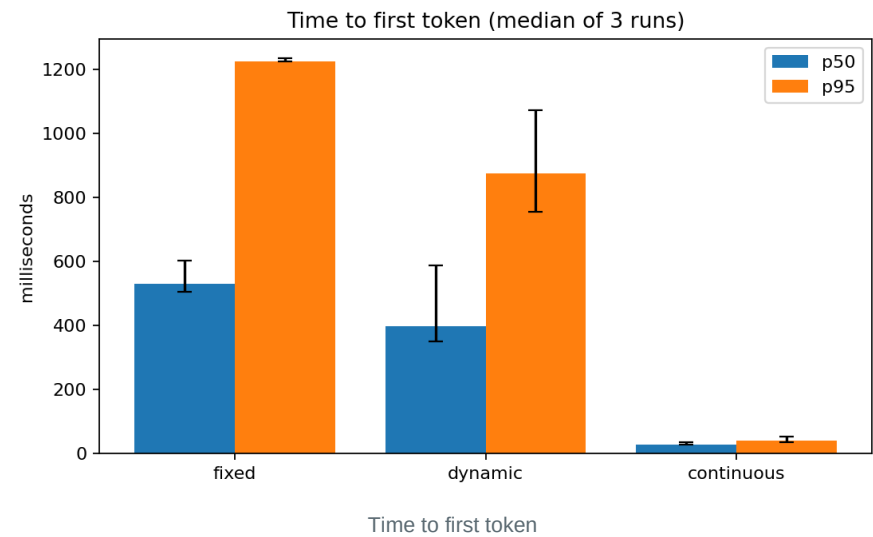
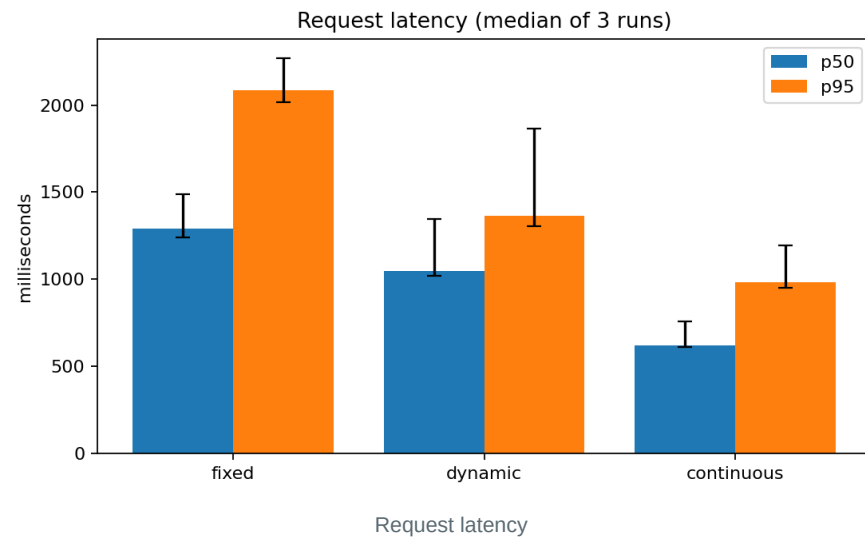
7. Sparse workload results

Strategy	Tokens/s	Latency p50	Latency p95	TTFT p50	TTFT p95	Avg. batch
Fixed	75.42	1290.17	2084.15	528.78	1225.97	2.50
Dynamic	76.36	1048.23	1365.51	396.34	873.98	1.46
Continuous	76.75	619.02	984.31	28.16	39.55	1.48

Sparse throughput was similar because arrivals dominated the makespan. Dynamic batching reduced p50 latency by 19% and p95 latency by 34% compared with fixed batching. Continuous admission reduced latency further.



Sparse latency detail



The dynamic timeout prevents a sparse request from waiting indefinitely for a full batch. Continuous batching gives the strongest TTFT because it can admit work whenever a slot opens.

8. Limitations and conclusion

This version performs real Qwen inference, but it recomputes active sequences on every token step with `use_cache=False`. It does not implement a production KV-cache manager, paged attention, or multi-GPU execution. Results apply to this model, hardware, workload, and parameter set.

The experiment demonstrates the intended trade-off without forcing one ranking. Fixed batching collects closed batches. Dynamic batching avoids indefinite waits under sparse arrivals. Continuous batching uses iteration-level admission to improve utilization and responsiveness.

Conclusion

On this setup, continuous batching performed best overall. Dynamic batching clearly improved sparse latency over fixed batching, which is the behavior the sparse workload was designed to reveal.